PE-JV-003

Aula 07 - Segurança em APIs & Defesa em Profundidade

Aquecimento

Kahoot

Revisão da Aula 06: Métricas RED vs USE, p99 contra Média, Cardinalidade no Prometheus, Correlação com Trace ID, Head contra Tail Sampling e LGPD.

Aguarde as orientações do professor.

O Desafio de Engenharia

Na última aula, instrumentamos Checkout, Pagamento e Estoque com OpenTelemetry sob **5.000 TPS** para suportar a Black Friday.

O incidente na Black Friday:
A telemetria estava toda verde: latência sob controle e zero erros 5xx. Porém, a segurança detectou um vazamento grave. Um comprador autenticado trocou o ID na URL da rota de pedidos e obteve endereço, dados cadastrais e comprovante de outro cliente em segundos. O identificador era um **UUID v4**.

O problema:
Logo depois, uma credencial de parceiro vazou na internet. O invasor chamou a rota de cancelamento. O serviço interno aceitou a chamada porque a assinatura do token era válida. Ninguém conferiu permissões ou escopos administrativos. Como blindar a API para validar a posse de cada objeto e barrar acessos indevidos no tráfego interno?

A Missão do Dia

No seu documento de System Design (Aula 07):

- Mapear a arquitetura de autenticação via **OAuth2 e OIDC** com Keycloak, detalhando emissão de JWT, checagem via JWKS e tempo de expiração.
- Definir a estratégia de autorização granular no nível do objeto (**BOLA**) e da função (**BFLA**), eliminando vetores de enumeração de recursos.
- Projetar o modelo **Zero Trust** para chamadas entre Checkout, Pagamento e Estoque, comparando credenciais de serviço com repasse de identidade.
- Mapear defesas contra o **OWASP Top 10** e **OWASP API Security Top 10**, priorizando falhas de controle de acesso e SSRF sob 5.000 TPS.

Sala de Aula Invertida: A Pesquisa

Acesse os materiais compartilhados e pesquise:

1. **BOLA contra BFLA:** Qual a diferença entre autorização no nível do objeto e no nível da função? Por que trocar ID sequencial por UUID não impede acessos indevidos?
2. **Validação Estrita de JWT:** Quais elementos de um token devem ser verificados obrigatoriamente a cada requisição? Por que tokens nunca devem trafegar segredos no corpo?
3. **Respostas HTTP e Enumeração:** Como a escolha do código de status HTTP pode transformar uma API em um oráculo de contas válidas? Como neutralizar essa enumeração?
4. **Zero Trust entre Microsserviços:** Por que a comunicação interna não pode confiar cegamente na rede do cluster?
5. **OWASP Top 10:2025:** Quais os principais vetores de invasão em APIs corporativas e como mitigar falhas de controle de acesso?

O Desafio Arquitetural

No seu documento de System Design, fundamente suas decisões técnicas:

1. **Autenticação, OIDC e Ciclo de Vida de Tokens:**
   - O que o gateway e os microsserviços precisam inspecionar em um token para garantir autenticidade, integridade e validade temporal antes de processar qualquer requisição?
   - Como balancear o tempo de vida de tokens em arquitetura stateless para mitigar o impacto de credenciais vazadas sem transformar cada chamada em consulta bloqueante ao banco?
2. **Autorização Granular e Tratamento de Respostas:**
   - Em qual camada do sistema deve ocorrer a validação de posse do recurso para impedir acesso a dados de outros compradores? Qual o risco de delegar essa checagem para a memória da aplicação?
   - Qual status HTTP o endpoint deve responder quando o registro solicitado existe mas pertence a outro titular? Qual status deve responder quando a credencial não tem permissão para a rota? Como essa escolha impede oráculos de enumeração?
3. **Zero Trust entre Microsserviços e Mitigação OWASP:**
   - Como garantir que chamadas internas entre microsserviços comprovem a identidade da aplicação chamadora sem presumir que a rede interna é segura? Quando repassar a identidade do cliente e quando atuar com credencial própria?
   - Como proteger o sistema contra falhas de controle de acesso e SSRF? Como desenhar defesas em profundidade sem degradar o throughput de 5.000 TPS?

Debate Técnico

É hora de debater com o comitê de engenharia (Staff Engineer):

- **Identificadores e Autorização:** Trocar IDs sequenciais por UUIDs elimina a necessidade de checar autorização no código? De onde surgem os identificadores em um ataque real?
- **O Retorno da API e Enumeração:** Se o registro solicitado existe no banco de dados, qual código de retorno protege a privacidade da plataforma sem revelar a existência do dado?
- **Validação da Assinatura:** O que acontece se a aplicação confiar nos parâmetros enviados no cabeçalho do próprio token para validar a assinatura?
- **Confidencialidade do Token:** O token JWT protege o sigilo das informações que transporta? Quais dados jamais devem ser incluídos nas claims?
- **Revogação em Arquitetura Desacoplada:** Se um usuário for desativado ou um token vazar, como interromper o acesso imediatamente sem perder as vantagens do modelo stateless?
- **Comunicação entre Microsserviços:** Se um serviço da malha for comprometido, o que impede chamadas indevidas aos demais microsserviços da rede interna?

Projeto Individual (System Design)

Cenário de Engenharia (Invasão de Pedidos na Black Friday):

- **Arquitetura Distribuída:** Checkout, Pagamento e Estoque processando **5.000 TPS** autenticados via token JWT emitido por Keycloak.
- **Incidente de BOLA:** Alteração de UUID na URL permite que compradores leiam pedidos alheios sem validação de posse no banco de dados.
- **Vazamento e Tráfego Interno:** Credencial vazada executa cancelamentos sem checagem de escopos na comunicação entre serviços.

Sua Missão:

- Responda às decisões do **Desafio Arquitetural** aplicadas diretamente ao cenário acima.
- Conduza o Threat Modeling e fundamente as defesas arquiteturais no debate técnico com a turma e o comitê.

Laboratórios Práticos (Opcional)

Disponibilizamos no LMS o guia de laboratórios e ferramentas oficiais da indústria:

- **Keycloak & Spring Security / Quarkus OIDC:** Configuração prática de autenticação OAuth2, emissão de JWT e validação de claims em Resource Server.
- **OWASP ZAP & Testes de Segurança:** Execução de varreduras automatizadas contra rotas REST para detecção de falhas de controle de acesso e headers inseguros.
- **Zero Trust & mTLS:** Simulação de comunicação segura serviço a serviço com certificados mútuos e políticas de autorização entre microsserviços.

Próxima Aula: Engenharia de Software com IA & Agentes

Nossa plataforma opera com segurança, observabilidade e resiliência em produção. O próximo passo é dominar como a inteligência artificial transforma o ciclo de engenharia de software e a construção de sistemas modernos.

Tópicos Conceituais:

- **Desenvolvimento Guiado por Especificação (SDD):** Como especificações rigorosas e contratos formais direcionam a geração de código com precisão e reduzem alucinações.
- **Arquitetura de Agentes e Padrões:** Anatomia de agentes autônomos, desenho de skills modulares, gerenciamento de memória e harnesses de avaliação contínua.
- **IA Embarcada e Ferramentas:** Uso de interfaces padronizadas para acoplamento de ferramentas, integração com Spring AI e guardrails contra comportamento estocástico.

Referências para Leitura Prévia:

- 📄 Anthropic: Building Effective Agents
- 📄 Model Context Protocol (MCP) Specification
- 📄 Thoughtworks: Spec-Driven Development
- 📄 Spring AI Reference Documentation

Referências da Aula 07 (Segurança & Zero Trust)

Links oficiais para estudo e aprofundamento:

- 📄 **OWASP Top 10:2025:** OWASP Top 10 Standard
- 📄 **OWASP API Security Top 10:** API Security Project (2023)
- 📄 **Boas Práticas de JWT:** RFC 8725: JSON Web Token BCP
- 📄 **OAuth 2.0 Framework:** RFC 6749: The OAuth 2.0 Authorization Framework
- 📄 **OpenID Connect Core:** OIDC Core 1.0 Specification
- 📄 **Arquitetura Zero Trust:** NIST SP 800-207 Zero Trust Architecture
- 📄 **Spring Security Resource Server:** Spring Security OAuth 2.0 Docs
- 📄 **Segurança e Privacidade:** LGPD, Lei 13.709/2018 (Planalto)

Obrig.ADA